

Application Flow Document v2.0

CloudWise AI

Version: 2.0

Status: Active Development

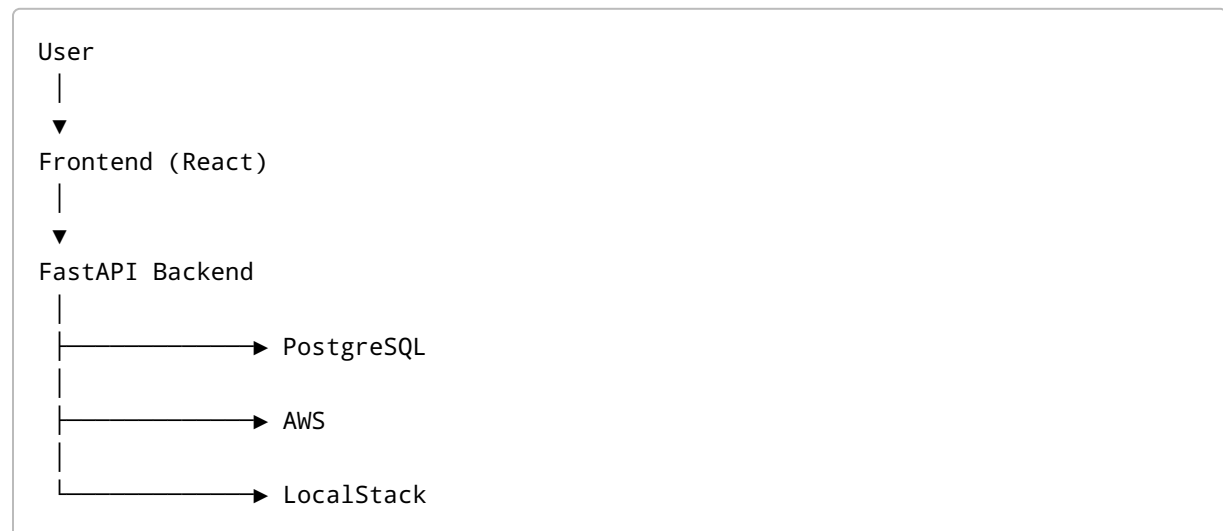
Last Updated: June 2026

1. Purpose

This document defines the complete application flow of CloudWise AI, including user interactions, frontend behavior, backend processing, database operations, AWS integrations, LocalStack integrations, recommendation generation, dashboard analytics, and future AI workflows.

The goal is to provide a clear understanding of how data moves through the system.

2. High-Level System Flow



3. User Authentication Flow

Registration Flow

```
User
↓
Signup Form
↓
Frontend Validation
↓
POST /auth/register
↓
Backend Validation
↓
Hash Password
↓
Store User
↓
Success Response
↓
Login Page
```

Login Flow

```
User
↓
Login Form
↓
POST /auth/login
↓
Verify Credentials
↓
Generate JWT
↓
Return Token
↓
Store Token
↓
Redirect Dashboard
```

Forgot Password Flow

```
User
↓
Forgot Password
↓
Enter Email
↓
POST /auth/forgot-password
↓
Generate Reset Token
↓
Send Email
↓
User Opens Link
↓
Reset Password Form
↓
POST /auth/reset-password
↓
Update Password
↓
Login
```

4. Cloud Connection Flow

CloudWise supports two environments:

AWS Environment

Real AWS Account

LocalStack Environment

Local Development Cloud

5. AWS Connection Flow

```
User
↓
Connect Cloud
↓
Enter AWS Credentials
↓
Submit
↓
POST /cloud/connect
↓
connect_aws()
↓
STS Validation
↓
Account Verification
↓
Save Cloud Account
↓
Success
```

6. LocalStack Connection Flow

```
User
↓
Enable LocalStack Toggle
↓
Frontend Sets

use_localstack=true

↓
POST /cloud/connect
↓
connect_aws()
↓
endpoint_url=http://localhost:4566
↓
LocalStack STS Validation
↓
Account Verification
```

↓
Success

7. Resource Discovery Flow

After cloud connection:

User
↓
Click Scan Resources
↓
POST /cloud/scan
↓
Cloud Scan Service

EC2 Discovery

discover_ec2()
↓
Describe Instances
↓
Normalize Data
↓
Create Resource Objects

Collected:

- Instance ID
- Instance Type
- Status
- Region
- Tags

EBS Discovery

discover_ebs()
↓
Describe Volumes

```
↓  
Normalize Data  
↓  
Create Resource Objects
```

Collected:

- Volume ID
- Size
- Attachments
- Encryption
- Status

8. Resource Persistence Flow

After discovery:

```
Discovered Resources  
↓  
Normalization  
↓  
ResourceInventory Mapping  
↓  
Database Insert  
↓  
resource_inventory
```

Stored Fields:

```
resource_id  
resource_name  
resource_type  
region  
status  
tags  
metadata_json
```

9. Metrics Collection Flow

```
Resource IDs
↓
CloudWatch Request
↓
CPU Utilization
↓
Metrics Processing
↓
metrics Table
```

Current Metric:

CPU Utilization

Future:

Memory
Disk
Network

10. Cost Analysis Flow

```
Cloud Account
↓
Cost Explorer
↓
Daily Costs
↓
Monthly Costs
↓
Cost Aggregation
↓
Dashboard Metrics
```

Generated Data:

- Current Spend

- Monthly Spend
- Cost Trends

11. Recommendation Engine Flow

Input Sources

Resources
+
Metrics
+
Costs



Recommendation Engine

Rule 1

Idle EC2 Detection

CPU < 5%



Recommendation

Shutdown Instance

Rule 2

Underutilized EC2

CPU < 20%



Recommendation

Resize Instance

Rule 3

Unattached EBS

No Active Attachments

↓

Recommendation

Delete Volume

12. Recommendation Storage Flow

```
Recommendation Engine
↓
Generate Recommendation
↓
Calculate Savings
↓
Store Recommendation
↓
recommendations Table
```

Stored Data:

```
priority
estimated_savings
description
recommendation_type
```

13. Dashboard Flow

Dashboard Load

User
↓
Dashboard Page
↓
GET /dashboard
↓
Backend Aggregation

Backend Reads:

resource_inventory
recommendations
metrics
cost_data

Backend Calculates:

Total Resources
Monthly Spend
Potential Savings
Health Score

↓

Response

↓

Frontend Dashboard

14. Dashboard Components Flow

KPI Cards

```
Dashboard API
↓
Total Resources

Dashboard API
↓
Monthly Spend

Dashboard API
↓
Potential Savings

Dashboard API
↓
Health Score
```

Cost Trend Chart

```
Cost Data
↓
Aggregation
↓
30 Day Trend
↓
Area Chart
```

Resource Distribution

```
Resource Inventory
↓
Group By Type
↓
Visualization
```

15. Resource Explorer Flow

```
Resources Page
↓
GET /cloud/resources
↓
Database Query
↓
Paginated Results
↓
Frontend Table
```

Features:

- Search
- Filter
- Sort

16. LocalStack Demo Flow

Purpose:

Prevent empty dashboards during development and demonstrations.

Flow

```
Start LocalStack
↓
Create Mock Resources
↓
Run Discovery
↓
Store Resources
↓
Generate Recommendations
↓
Populate Dashboard
```

Mock Resources:

- EC2
- EBS

17. Future Forecasting Flow

```
graph TD;
    A[Historical Costs] --> B[Forecast Engine];
    B --> C[Prediction Model];
    C --> D[7 Day Forecast];
    D --> E[30 Day Forecast];
    E --> F[90 Day Forecast];
```

18. Future AI Copilot Flow

```
graph TD;
    A[User Question] --> B[Copilot Interface];
    B --> C[Context Builder];
```

Context Sources:

- Resources
- Metrics
- Costs
- Recommendations
- Forecasts

↓

LLM Processing

↓

AI Response

Example:

Why is my AWS bill increasing?

19. Report Generation Flow

```
User
↓
Generate Report
↓
Collect Dashboard Data
↓
Collect Recommendations
↓
Generate Summary
↓
Create PDF
↓
Download Report
```

20. Error Handling Flow

AWS Failure

```
AWS Error
↓
Capture Exception
↓
Return User Friendly Message
```

LocalStack Failure

```
Connection Error
↓
Show Diagnostic Message
```

Database Failure

```
Database Error
↓
Log Error
↓
Return Safe Response
```

21. Complete End-to-End User Flow

```
Signup
↓
Login
↓
Connect AWS / LocalStack
↓
Validate Credentials
↓
Scan Resources
↓
Discover EC2 & EBS
↓
Store Resources
↓
Collect Metrics
↓
Analyze Costs
↓
Generate Recommendations
↓
Populate Dashboard
↓
View Insights
```

↓
Forecast Costs
↓
Generate Reports
↓
Optimize Infrastructure

Application Status

Current Production Features

- ✓ Authentication
- ✓ Forgot Password
- ✓ AWS Integration
- ✓ LocalStack Integration
- ✓ EC2 Discovery
- ✓ EBS Discovery
- ✓ Resource Inventory
- ✓ Dashboard Analytics
- ✓ Recommendation Engine

Planned Features

- 🔄 Forecasting Engine
- 🔄 AI Copilot
- 🔄 PDF Reports
- 🔄 S3 Discovery
- 🔄 RDS Discovery

Document Status:

Approved

Version:

2.0